

RateIt

Developer Handoff — MVP Architecture & Build Plan

Purpose

This document translates the RateIt concept into a buildable MVP for a developer or small technical team. It defines the first product boundary, the minimum system architecture, the core data objects, the extension behaviour, the admin workflow, and the staged build order.

MVP outcome

A browser extension that checks the current store domain against a reviewed merchant database and shows a trust verdict, a short explanation, and the top reasons before the shopper buys.

In scope

Manual or semi-manual merchant reviews, rules-based verdict logic, admin dashboard, API, browser extension popup, user reporting.

Out of scope

AI auto-detection, large-scale scraping, full merchant portal, public review marketplace, mobile app, complex analytics.

Build principle

Start with human-reviewed trust records and deterministic rules. Prove usefulness before automating collection or interpretation.

1. Product definition

RateIt is not a generic review app. The first product is a point-of-purchase trust signal. It appears while a shopper is on an online store and answers a simple question: should this buyer feel comfortable, cautious, or very cautious before paying?

Primary user stories

- As a shopper on an unfamiliar store, I want a fast verdict and a few reasons so I can decide whether to proceed.
- As a RateIt reviewer, I want to assess a merchant in a standard format so verdicts are disciplined and repeatable.
- As an admin, I want to update merchant records, moderate incoming reports, and trigger reassessments.

Verdict states

Verdict	Meaning	Default shopper message
Trusted	Strong positive signals and no serious red-flag pattern.	This store shows solid trust signals.
Caution	Mixed or incomplete trust profile. Buyer should slow down.	Some trust signals are missing. Review before buying.
High Risk	Multiple serious warnings or repeated harmful patterns.	Multiple warning signals detected. Proceed very carefully.
Insufficient Data	Not enough evidence yet to rate fairly.	This store has not yet been fully assessed.

2. MVP boundary

The MVP should stay narrow. It is better to work well for a limited set of merchants than to pretend to cover the whole internet.

Must exist in v1

- Merchant database with reviewed domains and current verdicts.
- Admin dashboard for merchant assessments, verdict updates, and report moderation.
- Rules-based scoring engine using the four trust pillars and red-flag triggers.
- API that returns verdict, pillar summary, top reasons, and metadata for a domain.
- Browser extension popup that displays the verdict and links to deeper detail or reporting.
- User report submission flow linked to a domain.

Can wait until later

- Automated scraping or AI-led merchant scoring.
- Mobile app and large public directory.

- Merchant self-service portal and billing.
- Advanced analytics, trend graphs, or social alert automation.
- Complex review feeds, public comments, or reputation gamification.

3. Core system architecture

The MVP has five core layers. Each one does a distinct job.

Layer	Purpose	What the developer builds
Merchant database	Stores reviewed domains, verdicts, pillar ratings, evidence notes, and review dates.	Schema/tables or collections for merchants, assessments, user reports, admins, and verdict history.
Admin dashboard	Internal control room for adding merchants, editing assessments, and reviewing reports.	Authenticated web app with merchant list, merchant detail, assessment editor, and report moderation screens.
Rules engine	Converts stored assessments into a verdict and public-facing reasons.	Deterministic logic based on pillar states, red flags, and confidence rules.
API layer	Bridge between extension, full report page, admin tools, and database.	Endpoints for domain lookup, full report retrieval, report submission, and admin updates.
Browser extension	User-facing trust signal at time of purchase.	Extension popup UI, domain detection, API call, verdict display, and reporting shortcut.

4. Data model

The exact implementation can vary, but the data model needs the following minimum objects.

Merchant

Minimum fields: id, store_name, domain, category, current_verdict, summary, last_reviewed_at, next_review_at, status

Assessment

Minimum fields: id, merchant_id, reviewer_id, transparency_rating, reliability_rating, integrity_rating, communication_rating, red_flag_severity, internal_rationale, public_verdict_line, public_top_reasons[], created_at

User report

Minimum fields: id, merchant_id or submitted_domain, report_type, description, evidence_url or attachment ref, contact_email (optional), status, created_at

Verdict history

Minimum fields: id, merchant_id, previous_verdict, new_verdict, changed_by, change_reason, changed_at

Admin user

Minimum fields: id, name, email, role, password_hash or auth provider id

5. Rules engine

Start with rules, not AI. The reviewer records pillar states and red flags; the rules engine converts that into a verdict.

Trust pillars

- Transparency — refund clarity, shipping clarity, pricing clarity, subscription disclosure, product description quality.
- Reliability — delivery confidence, fulfilment consistency, payment normality, refund behaviour after failed delivery.
- Integrity — honesty of claims, realism of discounts, absence of fake scarcity or fake local stock claims.
- Communication — real support pathway, visible business identity, response viability, usable contact options.

Initial roll-up rules

- Trusted: at least three pillars are Strong, no major red flag, and evidence confidence is adequate.
- Caution: mixed profile overall, one weak pillar or several meaningful gaps, but not enough for High Risk.
- High Risk: two or more weak pillars, or a severe red-flag cluster, or repeated harmful pattern.
- Insufficient Data: evidence too thin or not yet assessed enough to rate fairly.

Hard guardrails

- Do not assign Trusted without documented positive signals.
- Do not assign High Risk without written rationale and recorded red flags.
- Do not use Insufficient Data as a substitute for incomplete reviewer work if evidence was available.

6. API contract (plain-English endpoints)

The names can change, but the API needs these functions.

Method / route	Used by	Purpose
----------------	---------	---------

GET /merchant/by-domain? domain=example.com	Extension	Return current verdict, last reviewed date, pillar snapshot, and public top reasons.
GET /merchant/{id}/report	Full report page	Return detailed report content for a known merchant.
POST /reports	Extension / public form	Submit a consumer report linked to a domain or merchant.
GET /admin/merchants	Admin dashboard	List merchants with filters by verdict, status, or review date.
POST /admin/merchant	Admin dashboard	Create merchant record.
PUT /admin/merchant/{id}/assessment	Admin dashboard	Save or update assessment data and trigger verdict recalculation.

7. Browser extension behaviour

The extension should be simple enough for a shopper to understand in under five seconds.

Minimum extension flow

- Detect current domain when user is on a merchant site.
- Call the RateIt API with the domain.
- Receive verdict, verdict line, pillar snapshot, top reasons, and last reviewed date.
- Show a compact popup with the verdict first and reasons second.
- Offer a short path to View Full Report and Report This Store.

Popup content hierarchy

- RateIt Signal header
- Domain name
- Last reviewed date
- Large verdict label
- One-line verdict explanation
- Four-pillar snapshot
- Top reasons list
- Action buttons

Unknown domains

If the domain has no reviewed record, show Insufficient Data. Do not fake confidence. Offer Request Review or Report This Store as the next step.

8. Admin workflow

The admin dashboard can be plain and utilitarian. Its job is control, not marketing.

Minimum screens

- Login
- Merchant list / search
- Merchant detail page
- Assessment editor
- User reports review queue
- Verdict history / audit trail

Typical reviewer flow

- Find or create merchant record.
- Open assessment template and review the merchant.
- Set pillar ratings, red flags, rationale, and top public reasons.
- Save assessment and confirm verdict output.
- Schedule next review or flag for monitoring.

9. Build order

This is the recommended implementation sequence. It keeps the project disciplined and reduces wasted effort.

Phase	What gets built	Why first
Phase 1	Database schema + admin authentication + merchant CRUD.	Creates the data backbone and internal control room.
Phase 2	Assessment editor + rules engine + verdict storage.	Turns concept logic into a working decision system.
Phase 3	Domain lookup API + full report API.	Makes the data usable by other surfaces.
Phase 4	Browser extension popup + domain detection + report submission.	Creates the first real user-facing product.
Phase 5	Moderation tools, verdict history, and reassessment workflow polish.	Improves trust, control, and operational durability.

10. Acceptance criteria for MVP

- An admin can create and update a merchant assessment without touching code.
- A verdict is generated consistently from stored assessment data.

- The extension can identify a merchant by domain and display the current verdict with reasons.
- A shopper can submit a report tied to the current store.
- Unknown merchants are handled cleanly with Insufficient Data.
- Every verdict shown to a user can be traced back to a stored assessment and rationale.

11. Recommended implementation posture

Use a simple modern web stack that the team can ship quickly with. The specific framework matters less than the discipline of the system. Avoid over-engineering. The first win is not scale; it is proving that shoppers value a live trust signal before checkout and that the internal review workflow can support it.

Non-goals for v1

- Do not promise perfect scam detection.
- Do not frame the product as a legal adjudication engine.
- Do not build heavy automation before the manual review workflow is proven.
- Do not broaden coverage faster than review quality can support.

12. Handoff summary

RateIt v1 is a human-reviewed trust signal delivered through a browser extension. The system begins with a disciplined merchant database, a simple admin dashboard, deterministic verdict logic, and an API that feeds the extension. Build the control room first, then the verdict engine, then the shopper-facing extension. Keep the boundary narrow and the language defensible.